School of Computer Science

COMP 8347 “Internet Applications and Distributed Systems”

Summer 2026

Week 1
Internet Architectures. Client/Server Models. HTTP/REST Basics.

DR. OLENA SYROTKINA

University of Windsor

Copyright Notice

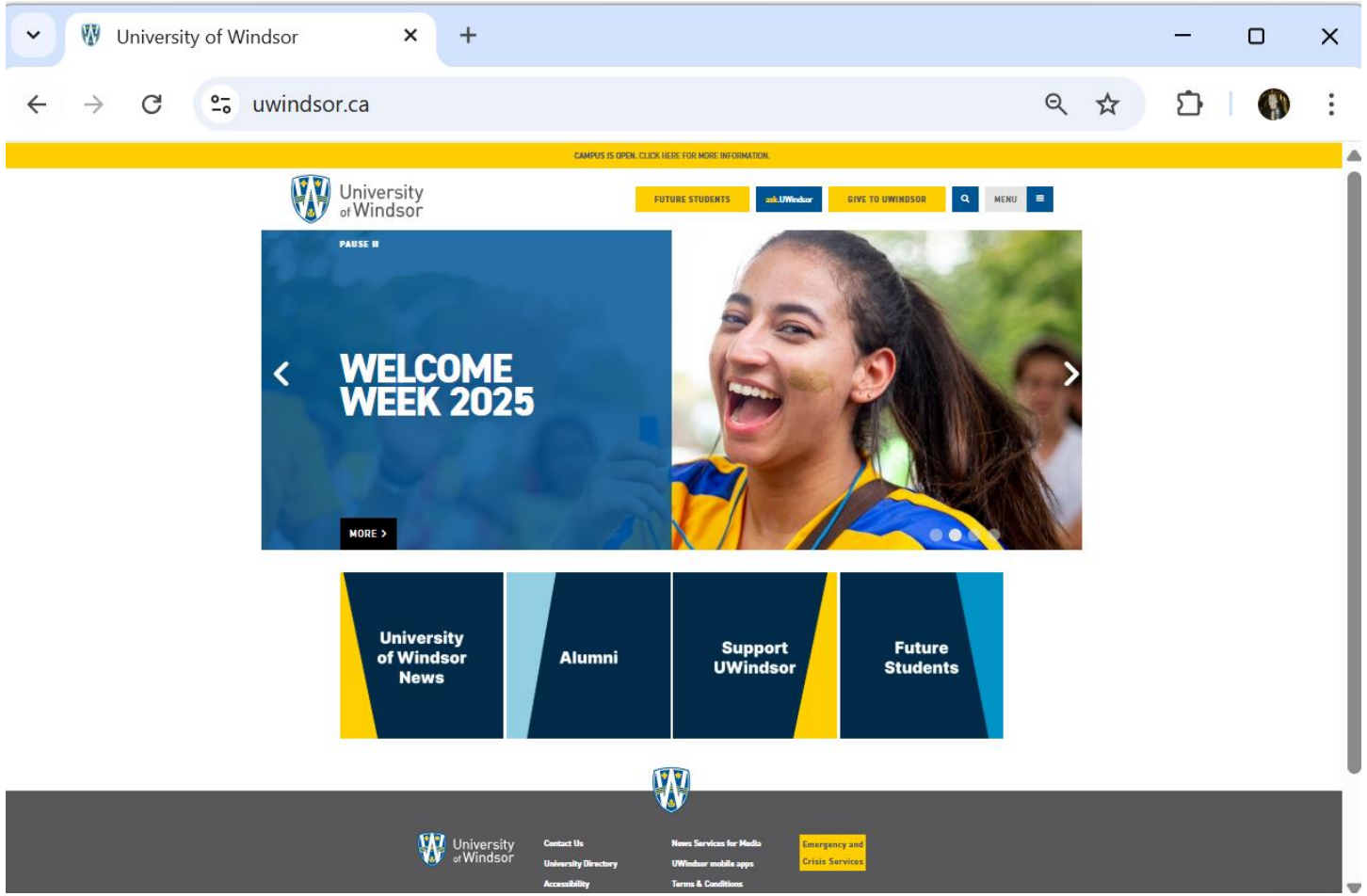
This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero, Chegg, StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- Internet & Client/Server Model
- DNS & IP Addresses
- Protocol Layers & TCP/IP
- What is HTTP?
- Client → Server → Database Flow
- Other Concepts: Cookies. Sessions. Caching. HTTPS.
- Tools & Observations.
- Introduction to Python.

The Internet & Client/Server Model

What actually happens when you type **uwindsor.ca** into your browser and press Enter?



University of Windsor

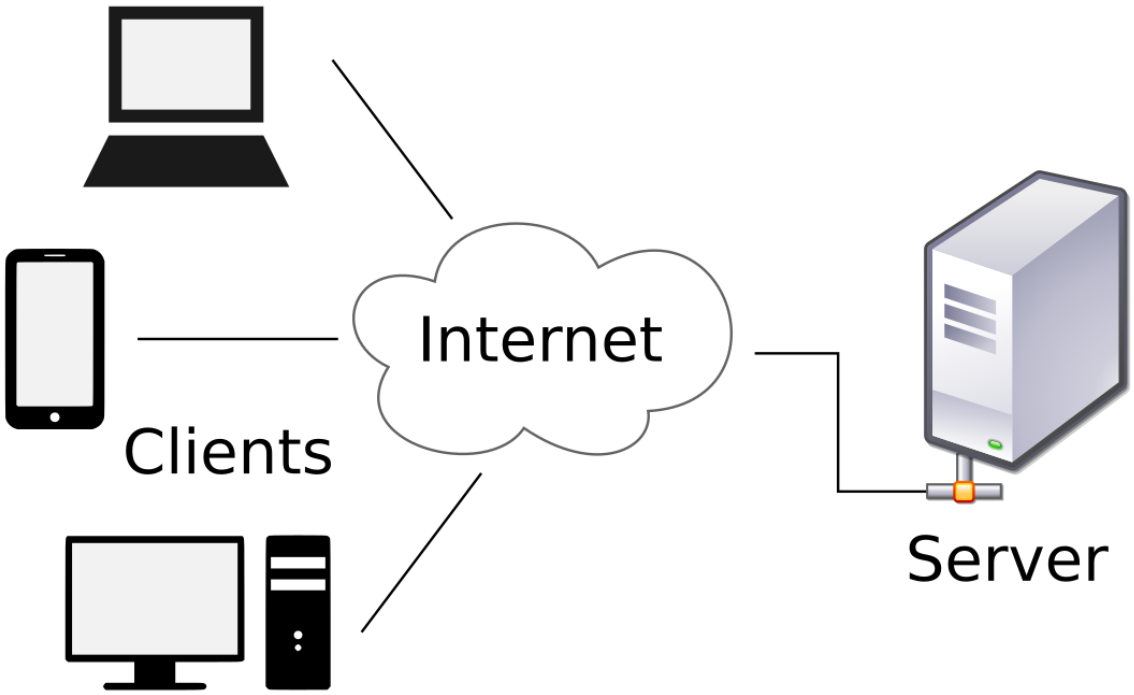
The Internet & Client/Server Model

What is the Internet?

The **internet** is nothing more than a giant network of networks. Millions of computers around the world are connected together, following common rules we call ***protocols***.

The Internet & Client/Server Model

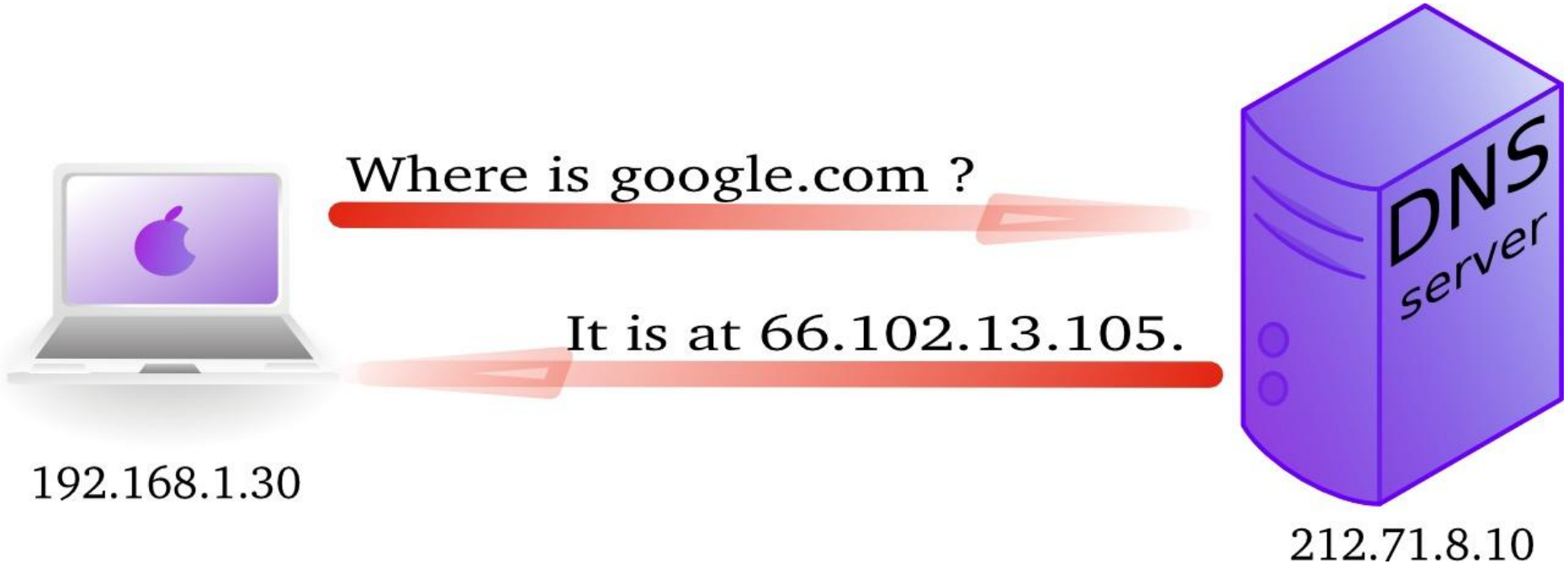
What is the **client–server model?**



The **client** is the device or program that requests resources – your browser, your mobile app.
The server is a machine that stores and delivers those resources.

Think of it like a restaurant:
You, the **customer**, are the **client**.
The **waiter** carries your order to the **kitchen**. The kitchen (the **server)** prepares the meal. The **waiter** then brings your meal back – that's the **response**.
This model works well because **one server can serve thousands, even millions, of clients at the same time.**

DNS & IP Addresses



DNS – the Domain Name System. DNS is like the phonebook of the internet. It translates human-friendly names into numeric IP addresses.

DNS & IP Addresses

Mini activity: Let's try this. If I run **`nslookup example.com`** in the terminal, it returns an IP address. That's the server's real "street address."

Command Prompt

```
Microsoft Windows [Version 10.0.26100.5074]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ellen>nslookup google.com
Server:  dns-vip1.uwindsor.ca
Address:  10.10.10.10

Non-authoritative answer:
Name:    google.com
Addresses:  2607:f8b0:400b:803::200e
          142.251.41.46


C:\Users\ellen>nslookup uwindsor.ca
Server:  dns-vip1.uwindsor.ca
Address:  10.10.10.10

Name:    uwindsor.ca
Address:  137.207.71.196
```

DNS works in steps:

1. Browser cache (have I looked this up recently?).
2. Operating system cache.
3. ISP's DNS server.
4. Root servers.
5. Authoritative servers for the domain.

Often the result is cached so the next lookup is faster.

And just a note: IPv4 addresses are 32-bit, about 4.3 billion possible addresses. That's not enough for our modern world, so we also have IPv6 – 128-bit, almost unlimited.

IPv4 vs IPv6

What is an IP Address?

An IP address is like the street address of a computer on the internet. It tells packets of data where to go. Without an address, your computer wouldn't know where to send requests, and servers wouldn't know where to reply. Every device on the internet must have a unique IP address.

IPv4 (Internet Protocol version 4)

- Created in the early days of the internet (1980s).
- Uses 32 bits → about 4.3 billion unique addresses.
- Written as four numbers (0–255) separated by dots.
- Example: 192.168.0.1

Still the most common, widely deployed protocol.

Problem: In the 1980s, 4 billion addresses seemed more than enough. But today we have billions of smartphones, laptops, IoT devices, sensors, cars, even smart fridges. We ran out.

IPv4 vs IPv6

IPv6 (Internet Protocol version 6)

- Introduced in the 1990s as a solution.
- Uses 128 bits → 340 undecillion addresses (that's 340 followed by 36 zeros!).
- Basically, unlimited addresses.
- Written in hexadecimal, separated by colons.
- Example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334
- Designed to replace IPv4, but adoption has been slow because it requires updating hardware and software.

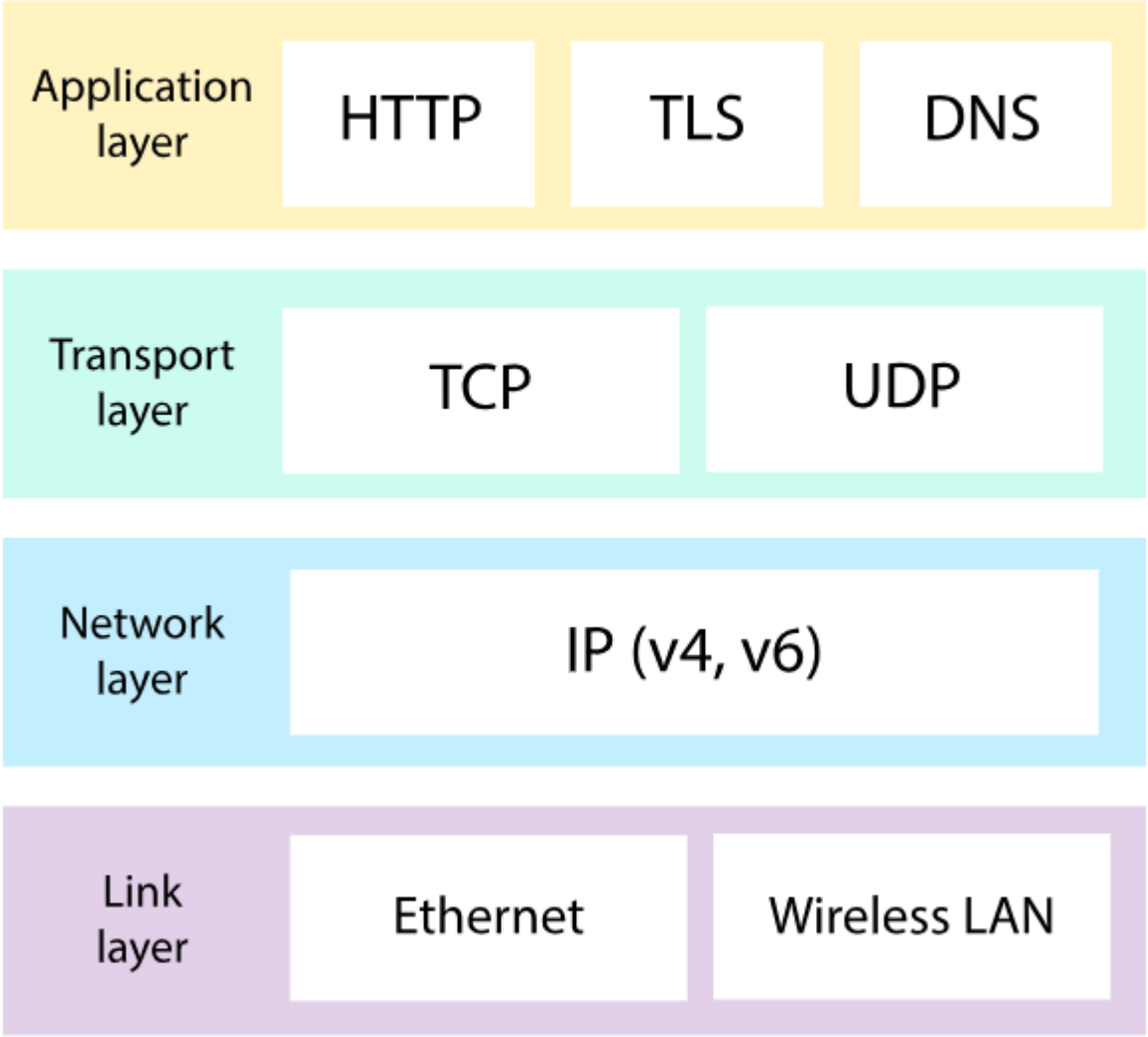
Why Both Still Exist

•Most of today's internet still runs on IPv4.

•IPv6 is gradually being rolled out alongside IPv4 – this is called **dual-stack** mode.

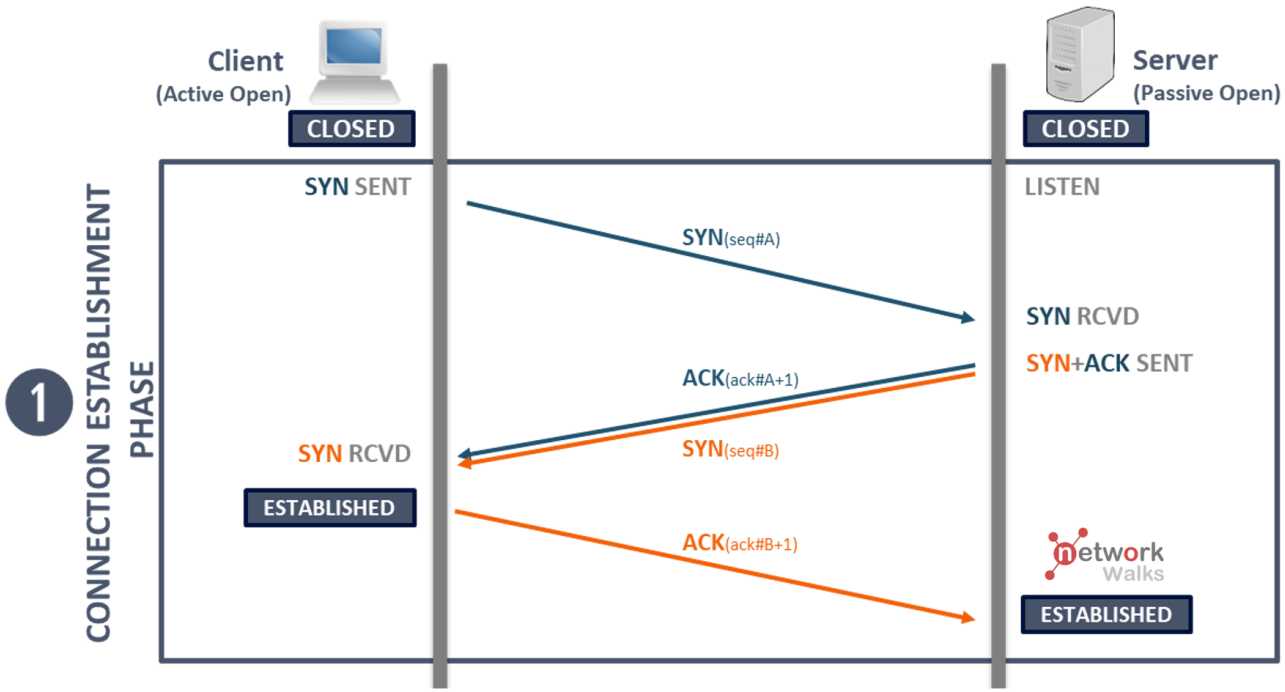
•Slowly, IPv6 will take over as more networks and devices support it.

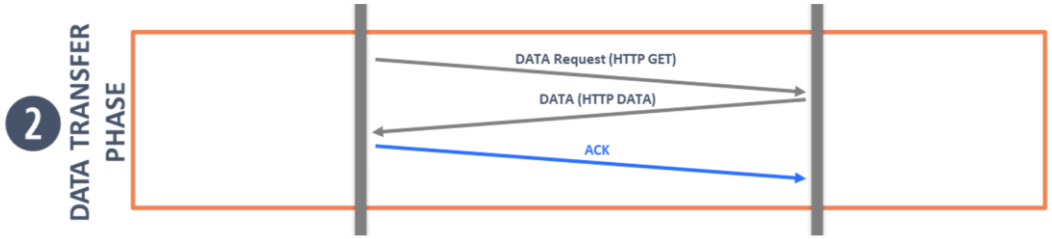
Protocol Layers & TCP/IP

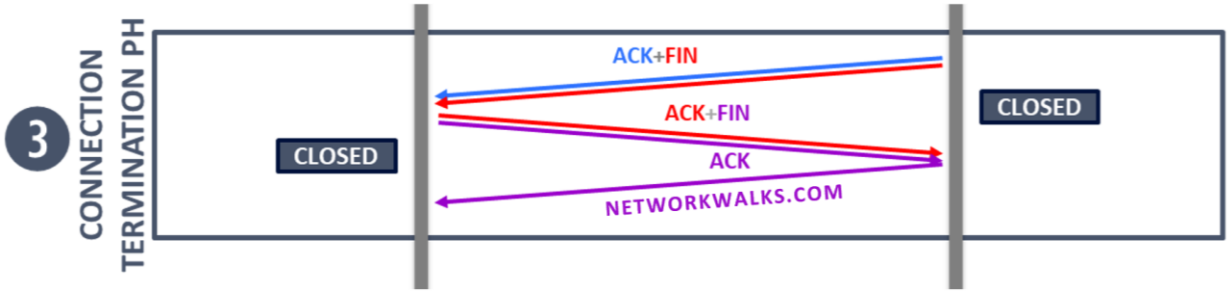


- **Link layer:** the physical network – Ethernet cables, Wi-Fi.
- **Network layer:** IP – Internet Protocol – the addressing system.
- **Transport layer:** TCP or UDP – ensures data is delivered.
- **Application layer:** HTTP, DNS, email protocols.

TCP 3-way Handshake Process







HTTP – The Language of the Web

Anatomy of a Request (Client → Server)

When your browser (the **client**) asks a web server for something (like a webpage), it sends an **HTTP request**. The main parts are:

1.Method

1. What action the client wants:
    1. GET → retrieve data (e.g., load a webpage).
    2. POST → send data (e.g., form submission).
    3. PUT → update existing data.
    4. DELETE → remove data.

2.Path

1. The specific resource requested, e.g., /products.

3.Headers

1. Extra information about the request, such as:
    1. Host: example.com → which server to contact.
    2. User-Agent: Chrome → what browser is making the request.
    3. Accept: text/html → what kind of response is expected.

4.Body (optional)

1. Contains data the client sends, e.g., login credentials, form submissions, JSON payloads.

HTTP – The Language of the Web

Anatomy of a Response (Server → Client)

The server answers with an **HTTP response**, which has:

Status Line

Tells whether the request succeeded. Example:

HTTP/1.1 200 OK

Headers

Metadata about the response:

Content-Type: text/html → response contains HTML.

Content-Length: 137 → size of the content.

Cookies and other info may also appear here.

Body

The actual content: HTML, JSON, image, video, etc.

HTTP – The Language of the Web

Anatomy of a Request

```
GET /products HTTP/1.1
Host: example.com
User-Agent: Chrome
Accept: text/html
```

- Method: GET, POST, PUT, DELETE.
- Path: /products.
- Headers: extra info.
- Body: optional, e.g., form submission.

Anatomy of a Response

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 137

<html> ... </html>
```

- Status line.
- Headers (content type, cookies).
- Body (HTML, JSON, image, etc.).

Status Codes

- 200 OK - success.
- 404 Not Found - page missing.
- 500 Internal Server Error - server crashed.
- 301 Moved Permanently - redirect.

Client → Server → Database

Most modern websites don't just serve static pages – they interact with databases.
Example: You log into Facebook.

1. Browser sends a POST request with your username and password.
2. The server checks the database: Is this valid?
3. If yes, server responds: Login successful.
4. Browser loads your personalized homepage.

So the flow is:
Client → Web Server → Application Logic → Database → Back.

Cookies. Sessions. Caching. HTTPS.

Cookies: small pieces of data stored in the browser, sent with every request.

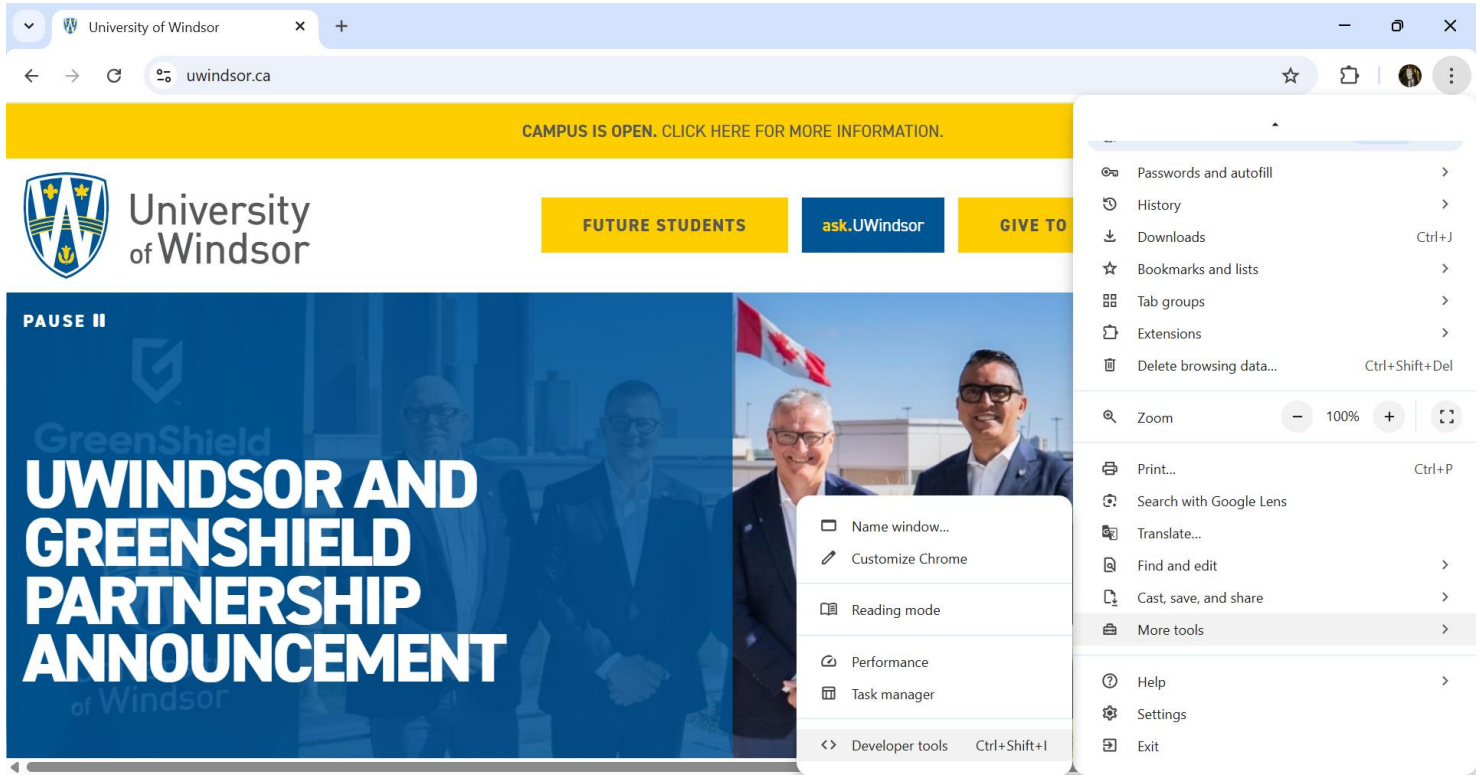
Sessions: stored on the server, linked to a session cookie.

Caching: storing responses so we don't always hit the server.

HTTPS: secure HTTP using TLS encryption – critical for banking, passwords, privacy.

Tools & Observations

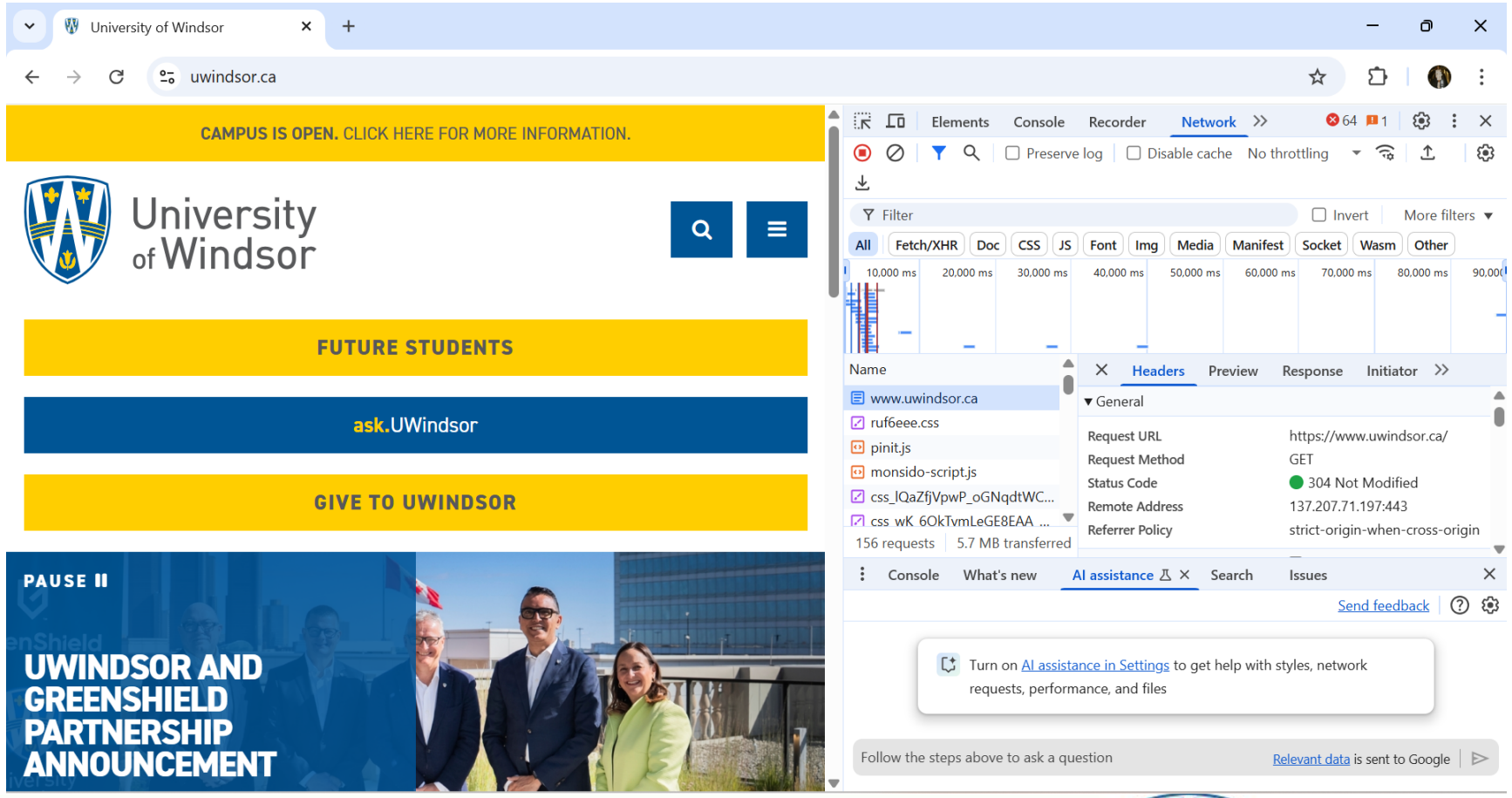
In your browser, open **Developer Tools → Network tab**. Every page, image, script – all requests are listed. encryption – critical for banking, passwords, privacy.



University of Windsor

Tools & Observations

In your browser, open **Developer Tools → Network tab**. Every page, image, script – all requests are listed. encryption – critical for banking, passwords, privacy.



Group Task

What happens when you type
https://www.linkedin.com/ ?

Name 6 steps….